Assignment 4 (RTOS)

1. Mention two challenges in designing real-time systems. Why is task prioritization critical in real-time systems?

Challenges in Designing Real-Time Systems

1. Strict Timing Constraints:
One of the biggest challenges in real-time system design is ensuring that every task completes within its specified deadline. In real-time systems, correctness depends not only on the logical output but also on the time at which the output is produced. Missing deadlines in applications such as avionics, automotive control systems, or medical devices can lead to catastrophic failures.

2. Limited System Resources:
Real-time systems usually operate under limited hardware resources such as CPU speed, memory, power, and I/O bandwidth. Designers must ensure that multiple time-critical tasks share these limited resources efficiently without causing delays or deadline misses.

Importance of Task Prioritization

Task prioritization is critical in real-time systems because tasks differ in urgency and importance. High-priority tasks often have strict deadlines and must be executed before lower-priority tasks. Proper prioritization ensures predictable system behavior, guarantees timely execution of critical tasks, reduces response time, and prevents system failure due to missed deadlines. It also helps the scheduler make efficient decisions under heavy workload conditions.

2. How do hardware constraints affect real-time system design?

Hardware constraints play a crucial role in real-time system design. Limited processor speed restricts how many tasks can be executed concurrently and affects scheduling decisions. Limited memory impacts the size of tasks, stacks, buffers, and data structures used by the RTOS. I/O constraints, such as slow sensors or communication interfaces, introduce delays that must be accounted for in timing analysis. Power constraints are especially important in embedded and portable real-time systems, requiring low-power processors and energy-efficient task execution. These constraints force designers to optimize code, select efficient algorithms, and carefully balance performance with timing requirements.

3. Explain the importance of deadline management in real-time systems.

notes.arpanadhikari7.com.np

Deadline management is a fundamental requirement of real-time systems. It ensures that tasks complete their execution within predefined time limits. In hard real-time systems, missing a deadline may result in total system failure, whereas in soft real-time systems, performance degradation may occur. Proper deadline management allows tasks to be classified as hard, firm, or soft real-time tasks. It also helps the system detect missed deadlines, trigger corrective actions, and maintain system stability. Effective deadline management improves system predictability, reliability, fault tolerance, and ensures compliance with real-time constraints.

4. What are the primary goals of task scheduling in an RTOS?

The primary goals of task scheduling in an RTOS include ensuring that all time-critical tasks meet their deadlines while efficiently utilizing system resources. The scheduler aims to minimize response time for high-priority tasks and reduce latency for external events. Predictable and deterministic behavior is a key goal, enabling accurate timing analysis. The scheduler also prevents task starvation by managing priorities appropriately. Additionally, it seeks to maximize CPU utilization without violating real-time constraints and to maintain system stability under varying workloads.

Available on notes.arpanadhikari7.com.np

5. Explain the difference between preemptive and cooperative scheduling.

Difference between Preemptive and Cooperative Scheduling:

Basis	Preemptive Scheduling	Cooperative Scheduling
Definition	RTOS can interrupt a running task to execute a higher-priority task	Tasks voluntarily release the CPU
CPU Control	Controlled by the scheduler	Controlled by the running task
Response Time	Very fast response to critical events	Slower response to urgent tasks
Predictability	Highly predictable and deterministic	Less predictable
Overhead	Higher due to context switching	Lower overhead
Usage	Common in real-time systems	Used in simple or low-cost systems
Risk	Risk of priority inversion	Risk of CPU hogging

6. Define the terms latency, jitter, and response time.

Latency:
Latency refers to the delay between the occurrence of an external event (such as an interrupt) and the start of its processing by the system.
Jitter:
Jitter is the variation in timing between successive executions or responses of a task. In real-time systems, low jitter is important to ensure consistent and predictable behavior.
Response Time:
Response time is the total time taken from the arrival of an event to the completion of the task associated with that event, including waiting time, execution time, and any preemption delays.

7. What is a priority-based scheduling algorithm, and how does it work in an RTOS?

A priority-based scheduling algorithm assigns a priority level to each task based on its urgency and importance. In an RTOS, the scheduler always selects the highest-priority task that is in the ready state. If a higher-priority task becomes ready while a lower-priority task is executing, the scheduler immediately preempts the running task. This mechanism ensures fast response to critical tasks, predictable execution order, and efficient deadline management. Priority-based scheduling is widely used in real-time systems due to its simplicity and effectiveness in meeting timing constraints.

8. What is task context switching, and why is it important in RTOS?

notes.arpanadhikari7.com.np

Task context switching is the process of saving the state of a currently executing task and restoring the state of another task so that the CPU can switch execution between them. The task context includes CPU registers, program counter, stack pointer, and other processor state information.

Context switching is important in an RTOS because it enables multitasking and ensures that high-priority tasks receive immediate CPU access when required. It allows the system to respond quickly to interrupts and external events. Efficient context switching improves system responsiveness, helps meet deadlines, and ensures predictable real-time behavior. Although context switching introduces overhead, it is essential for supporting preemptive scheduling and real-time task management.

Available on notes.arpanadhikari7.com.np

9. Describe the impact of task starvation and ways to prevent it in an RTOS.

Task starvation occurs when a low-priority task is prevented from executing for an extended period because higher-priority tasks continuously occupy the CPU. This can cause system imbalance and may result in missed deadlines for non-critical tasks.

The impact of task starvation includes poor system fairness, delayed task execution, and potential system instability. In extreme cases, starved tasks may never execute, leading to functional failures.

Ways to prevent task starvation include priority aging, where the priority of waiting tasks is gradually increased over time. Time slicing ensures that tasks of equal priority share CPU time fairly. Proper priority assignment and load balancing also help reduce starvation. Some RTOS implementations use hybrid scheduling policies to maintain fairness while preserving real-time constraints.

10. Compare fixed-priority preemptive scheduling with dynamic-priority scheduling in real-time systems.

Basis	Fixed-Priority Preemptive Scheduling	Dynamic-Priority Scheduling
Priority Assignment	Priorities are assigned at design time	Priorities change during runtime
Flexibility	Less flexible	More flexible and adaptive
Scheduling Decision	Based on static priority values	Based on deadlines or system state
Predictability	Highly predictable	Less predictable
Overhead	Low scheduling overhead	Higher computational overhead
Examples	Rate Monotonic Scheduling (RMS)	Earliest Deadline First (EDF)
Usage	Widely used in hard real-time systems	Used in systems with varying workloads

11. How does an RTOS ensure fairness in scheduling?

An RTOS ensures fairness in scheduling by distributing CPU time among tasks in a controlled manner. Time slicing allows tasks of equal priority to share CPU execution in fixed time intervals. Priority aging increases the priority of tasks that have been waiting for a long time, preventing starvation.

Some RTOSs use round-robin scheduling for tasks with equal priority to ensure balanced CPU usage. Fair scheduling mechanisms ensure that while critical real-time constraints are met, lower-priority tasks are not indefinitely blocked, maintaining overall system stability and responsiveness.

12. Define task priority and explain its significance in meeting deadlines.

Task priority is a numerical or logical value assigned to a task that determines its importance relative to other tasks. In an RTOS, higher-priority tasks are scheduled for execution before lower-priority tasks.

Task priority is significant in meeting deadlines because tasks with strict timing constraints are given higher priority to ensure immediate CPU access. Proper priority assignment minimizes response time, ensures predictable execution, and helps guarantee that time-critical tasks meet their deadlines. Incorrect priority assignment may result in deadline misses and system failure.

Available on notes.arpanadhikari7.com.np

13. What are task dependencies, and how are they managed in RTOS?

Task dependencies occur when one task relies on the completion or output of another task before it can execute. These dependencies are common in real-time systems where tasks share data or resources.

RTOSs manage task dependencies using synchronization mechanisms such as semaphores, mutexes, event flags, and message queues. These mechanisms ensure proper task coordination, prevent race conditions, and maintain data consistency. Priority inheritance is often used to avoid priority inversion problems when tasks share resources.

14. What strategies can be used to handle tasks that overrun their allocated execution time? How does a scheduler handle tasks with equal priority?

Tasks that overrun their allocated execution time can cause deadline misses and system instability. Strategies to handle overruns include setting execution time limits, using watchdog timers to detect overruns, and terminating or restarting faulty tasks. The scheduler may lower the priority of overrunning tasks or switch the system to a safe state in safety-critical applications. Admission control is also used to prevent system overload.

When tasks have equal priority, the scheduler typically uses round-robin or time-sliced scheduling. Each task is given a fixed time quantum, ensuring fair CPU sharing and preventing any single task from monopolizing the processor.

15. You are designing an embedded system with limited memory and CPU resources. How would you optimize task scheduling?

To optimize task scheduling in an embedded system with limited resources, tasks should be carefully prioritized based on their deadlines and criticality. The number of tasks should be minimized, and unnecessary background tasks should be eliminated. Lightweight scheduling algorithms with low overhead should be selected.

Memory usage can be optimized by using static task allocation and avoiding dynamic memory allocation. Task execution time should be reduced through efficient algorithms and optimized code. Combining related tasks, reducing context switching, and selecting an appropriate RTOS configuration also help ensure efficient scheduling while meeting real-time constraints.

16. How would you handle synchronization between tasks and interrupts in a time-critical embedded application? Why is task synchronization more challenging in embedded systems with limited resources?

In a time-critical embedded application, synchronization between tasks and interrupts is handled using lightweight and deterministic mechanisms. Interrupt Service Routines (ISRs) are kept as short as possible and usually signal tasks using semaphores, event flags, or message queues instead of performing heavy processing. Deferred interrupt handling is commonly used, where the ISR notifies a high-priority task to complete processing. Critical sections are protected using mutexes or by temporarily disabling interrupts for very short durations.

Available on notes.arpanadhikari7.com.np

Task synchronization is more challenging in embedded systems with limited resources because memory, CPU power, and synchronization primitives are restricted. Excessive use of locks increases overhead and latency. Limited stack and heap size restrict the number of synchronization objects. Designers must carefully balance responsiveness and resource usage to avoid deadlocks, priority inversion, and missed deadlines.

17. What factors influence task priority assignment in an embedded system? Explain how scheduling algorithms are optimized for power consumption in embedded devices.

Task priority assignment in an embedded system is influenced by several factors. The most important factor is the **deadline requirement**, where tasks with tighter deadlines receive higher priority. Task criticality, execution frequency, response time requirements, and dependency on external events also influence priority assignment. Hardware interaction tasks such as sensor reading or actuator control are often given higher priority. System safety and fault tolerance requirements further guide priority decisions.

Scheduling algorithms are optimized for power consumption by reducing unnecessary CPU activity. Techniques such as dynamic voltage and frequency scaling (DVFS) adjust processor speed based on workload. Idle task scheduling allows the CPU to enter low-power sleep modes when no tasks are ready. Tickless scheduling reduces periodic timer interrupts. Power-aware schedulers group tasks to minimize frequent wake-ups, thereby extending battery life in embedded devices.

18. What are the key challenges of task scheduling in embedded systems compared to general-purpose systems?

Task scheduling in embedded systems faces stricter constraints than in general-purpose systems. Embedded systems must meet deterministic timing deadlines, whereas general-purpose systems focus mainly on throughput and user experience. Limited CPU speed, memory, and power resources make scheduling decisions more critical. Embedded systems often interact directly with hardware, requiring fast interrupt handling and predictable response times. Additionally, embedded systems usually lack virtual memory and advanced hardware support, making scheduling design more complex and error-prone.

notes.arpanadhikari7.com.np

19. How does an RTOS differ from a general-purpose operating system (GPOS)?

Basis	RTOS	GPOS
Primary Goal	Meet timing deadlines	Maximize throughput and usability
Predictability	Highly deterministic	Non-deterministic
Scheduling	Priority-based, real-time	Time-sharing, fairness-based
Latency	Very low and bounded	Variable and higher
Resource Usage	Lightweight	Resource-intensive
Examples	FreeRTOS, VxWorks	Windows, Linux

An RTOS is designed for time-critical applications, whereas a GPOS is designed for general computing tasks.

Available on notes.arpanadhikari7.com.np

20. What are the key features of an RTOS? Define the term "real-time" in the context of operating systems. What are the primary components of an RTOS?

The key features of an RTOS include deterministic scheduling, low interrupt latency, fast context switching, priority-based task management, and reliable inter-task communication. An RTOS is lightweight and optimized for embedded systems.

In the context of operating systems, **real-time** refers to the ability of the system to guarantee that tasks will respond within specified timing constraints. Correctness depends on both logical correctness and timing correctness.

The primary components of an RTOS include the kernel, scheduler, task management module, interrupt handler, memory management unit, inter-task communication mechanisms, and synchronization primitives such as semaphores and mutexes.

21. Explain the concept of a task or thread in an RTOS.

A task or thread in an RTOS is the smallest unit of execution that can be scheduled by the kernel. Each task has its own context, including program counter, stack, registers, and priority. Tasks can be periodic, aperiodic, or event-driven depending on application requirements.

RTOS tasks execute concurrently and are managed by the scheduler based on priority. Tasks communicate and synchronize using RTOS services such as message queues, semaphores, and event flags. Proper task design is essential to meet real-time deadlines and ensure system reliability.

22. How does an RTOS ensure predictability in task execution?

An RTOS ensures predictability by using deterministic scheduling algorithms, usually priority-based preemptive scheduling. Worst-case execution time (WCET) analysis is used to estimate task execution bounds. Interrupt latency and context switching time are kept minimal and bounded.

Fixed task priorities, controlled resource sharing, and avoidance of unbounded delays help maintain predictable behavior. RTOS kernels are designed to be small and efficient, ensuring that timing behavior remains consistent under varying workloads.

23. What is task prioritization, and why is it critical in an RTOS?

Task prioritization is the process of assigning priority levels to tasks based on their urgency and importance. In an RTOS, the scheduler always selects the highest-priority ready task for execution.

Task prioritization is critical because it ensures that time-critical tasks meet their deadlines. It reduces response time for urgent tasks, improves predictability, and prevents system failure. Without proper prioritization, lower-importance tasks could delay critical operations, leading to missed deadlines and unreliable system behavior.

Available on notes.arpanadhikari7.com.np

24. What are the different states a task can be in FreeRTOS?

In FreeRTOS, a task can exist in several well-defined states depending on its execution and scheduling status.

The **Running state** is when the task is currently executing on the CPU. At any given time, only one task can be in the running state on a single-core processor.

The **Ready state** includes tasks that are able to run but are not currently executing because another task of equal or higher priority is running.

The **Blocked state** occurs when a task is waiting for an event such as a semaphore, mutex, message, or a delay period to expire. Blocked tasks do not consume CPU time.

The **Suspended state** is when a task is explicitly suspended by the application and will not run until it is resumed manually.

The **Deleted state** is when a task has been removed from the system and its resources are freed. These states allow FreeRTOS to manage multitasking efficiently and predictably.

25. Which RTOS (FreeRTOS or VxWorks) is better suited for large-scale industrial applications, and why? Provide an example of FreeRTOS and VxWorks.

VxWorks is generally better suited for large-scale industrial applications due to its high reliability, certification support, and advanced features. It provides strong support for safety-critical standards such as DO-178C and IEC 61508, making it suitable for aerospace, defense, and industrial automation.

FreeRTOS, on the other hand, is lightweight, open-source, and ideal for small to medium embedded systems with limited resources. It is widely used in IoT and microcontroller-based applications.

An example of **FreeRTOS** usage is in IoT devices such as smart sensors and ESP32-based systems.
An example of **VxWorks** usage is in aerospace systems like the Mars Rover and industrial control systems.

notes.arpanadhikari7.com.np

26. Explain the architecture of FreeRTOS and VxRTOS.

The architecture of **FreeRTOS** is simple and lightweight. It consists of a small kernel that provides task management, scheduling, inter-task communication, and synchronization services. The kernel is highly configurable and modular, allowing developers to include only required features. It typically runs directly on microcontrollers without memory protection.

The architecture of **VxWorks (VxRTOS)** is more advanced and layered. It includes a real-time kernel, hardware abstraction layer, networking stack, file systems, memory management, and optional user-space separation. VxWorks supports both real-time tasks and processes, making it suitable for complex industrial and safety-critical systems.

Available on notes.arpanadhikari7.com.np

27. How does an RTOS ensure fairness in scheduling?

An RTOS ensures fairness in scheduling by using controlled scheduling policies that balance real-time constraints with equitable CPU sharing. Time slicing allows tasks of equal priority to share CPU time in fixed intervals. Round-robin scheduling ensures that no task monopolizes the CPU.

Priority aging is used to gradually increase the priority of tasks that have been waiting for a long time, preventing starvation. While high-priority tasks are favored to meet deadlines, fairness mechanisms ensure that lower-priority tasks still get execution time, maintaining system stability and responsiveness.

28. What are the reasons that cause the deadlock?

Deadlock occurs when a set of tasks are permanently blocked because each task is waiting for a resource held by another task. The main reasons that cause deadlock are based on four necessary conditions.

Mutual exclusion, where resources cannot be shared.
Hold and wait, where tasks hold resources while waiting for others.
No preemption, where resources cannot be forcibly taken from a task.
Circular wait, where tasks form a circular chain of resource dependency.

When all these conditions occur simultaneously, deadlock arises, halting system progress.

29. What is Priority Inversion? What are the factors that will affect the priority? What are the steps that are taken in priority inversion? What would have been the long-term impact if the priority inversion had not been resolved?

Priority inversion occurs when a high-priority task is blocked by a lower-priority task holding a required resource, while medium-priority tasks preempt the lower-priority task. This causes the high-priority task to wait indefinitely.

Factors that affect priority include task urgency, resource sharing, scheduling policy, and system load. Shared resources and improper synchronization are major contributors.

notes.arpanadhikari7.com.np

To handle priority inversion, techniques such as **priority inheritance** are used, where the low-priority task temporarily inherits the priority of the high-priority task. Another method is **priority ceiling protocol**, which limits resource access based on predefined priority levels.

If priority inversion is not resolved, it can cause missed deadlines, system instability, reduced predictability, and catastrophic failure in safety-critical systems such as aerospace or medical devices.

30. Why is it necessary to overcome deadlock? What are the different techniques to avoid deadlock?

It is necessary to overcome deadlock because deadlocked tasks cannot make progress, leading to system freeze, missed deadlines, and potential system failure. In real-time systems, deadlock can be catastrophic, especially in safety-critical applications.

Available on notes.arpanadhikari7.com.np

Different techniques to avoid deadlock include **deadlock prevention**, where one of the four deadlock conditions is eliminated. **Deadlock avoidance** uses careful resource allocation strategies to prevent circular wait. **Resource ordering** ensures that resources are acquired in a fixed order. **Timeout mechanisms** release resources if a task waits too long. In some systems, **deadlock detection and recovery** techniques are used to identify and resolve deadlocks dynamically.

31. Priority Ceiling Protocol vs Priority Inheritance.

Basis	Priority Inheritance Protocol (PIP)	Priority Ceiling Protocol (PCP)
Basic Idea	Low-priority task inherits priority of highest waiting task	Resource has a predefined priority ceiling
When Applied	Only when priority inversion occurs	Applied before resource access
Priority Change	Temporary and dynamic	Predefined and controlled
Deadlock Prevention	Does not prevent deadlock	Prevents deadlock
Complexity	Simple to implement	More complex
Overhead	Lower	Slightly higher
Usage	Widely used in RTOS	Used in safety-critical systems

Priority Inheritance resolves priority inversion dynamically, while Priority Ceiling prevents it proactively and avoids deadlocks, making PCP more suitable for hard real-time systems.

32. You are designing an embedded system with limited memory and CPU resources. How would you optimize task scheduling?

To optimize task scheduling in a resource-constrained embedded system, tasks should be carefully prioritized based on deadlines and criticality. Unnecessary background tasks should be eliminated to reduce CPU load. Lightweight scheduling algorithms such as fixed-priority preemptive scheduling are preferred due to low overhead.

Static task allocation should be used to avoid dynamic memory usage. Task execution time should be minimized using efficient algorithms and optimized code. Context switching should be reduced by combining related tasks where possible. Idle time should be utilized for power-saving modes. These strategies ensure efficient CPU usage while meeting real-time constraints.

notes.arpanadhikari7.com.np

33. How would you handle synchronization between tasks and interrupts in a time-critical embedded application?

In time-critical embedded applications, synchronization between tasks and interrupts is handled using lightweight and deterministic mechanisms. Interrupt Service Routines (ISRs) are kept minimal and are used only to signal tasks using semaphores, event flags, or message queues.

Deferred interrupt handling is used where heavy processing is shifted to high-priority tasks. Critical sections are protected using mutexes or by briefly disabling interrupts. Care is taken to avoid blocking calls inside ISRs. This approach ensures fast response, low latency, and predictable system behavior.

Available on notes.arpanadhikari7.com.np

34. What factors influence task priority assignment in an embedded system?

Task priority assignment is influenced by deadline requirements, task criticality, and response time constraints. Tasks with strict deadlines or safety-critical functions are given higher priority. Task execution frequency, interaction with hardware, and dependency on external events also affect priority.

System constraints such as CPU speed, memory availability, and power consumption further influence priority decisions. Improper priority assignment can lead to missed deadlines, priority inversion, or system instability.

35. How do you ensure that inter-task communication does not result in task starvation?

Task starvation due to inter-task communication is prevented by using fair scheduling mechanisms. Time slicing ensures equal CPU sharing among tasks of the same priority. Priority aging gradually increases the priority of waiting tasks to prevent indefinite blocking.

Non-blocking communication mechanisms and bounded wait times are used. Proper priority assignment and avoiding excessive use of locks also reduce starvation. These techniques ensure balanced communication and system fairness.

36. Describe a scenario where task synchronization fails, and discuss how to debug and fix it.

A task synchronization failure can occur when two tasks access shared data without proper locking, leading to race conditions. For example, one task updates shared data while another reads it simultaneously, causing inconsistent results.

Debugging involves enabling RTOS trace tools, logging task states, and monitoring semaphore or mutex usage. Fixing the issue requires proper use of mutexes, semaphores, or critical sections. Priority inversion issues can be addressed using priority inheritance. Correct synchronization restores data consistency and system reliability.

notes.arpanadhikari7.com.np

37. If two tasks are deadlocked over a resource, how does an RTOS detect and resolve the deadlock?

An RTOS may detect deadlock by monitoring task states and resource ownership, identifying circular wait conditions. Some systems use timeout mechanisms to detect tasks waiting indefinitely.

Deadlock resolution techniques include forcefully releasing resources, restarting tasks, or using priority inheritance. In real-time systems, deadlock prevention techniques such as resource ordering or priority ceiling protocols are preferred to avoid runtime failures.

Available on notes.arpanadhikari7.com.np

38. Compare the overhead of different synchronization mechanisms (e.g., mutexes vs semaphores).

Aspect	Mutex	Semaphore
Overhead	Lower	Higher
Ownership	Owned by a single task	No ownership
Priority Inversion Handling	Supports priority inheritance	Usually does not
Complexity	Simple	More complex
Usage	Resource protection	Signaling and counting
Suitability	Critical sections	Event synchronization

Mutexes have lower overhead and are ideal for protecting shared resources, while semaphores offer flexibility at the cost of higher overhead.

39. Discuss trade-offs between fairness, latency, and complexity in task synchronization.

Ensuring fairness may increase latency as CPU time is shared among tasks. Reducing latency often requires prioritizing critical tasks, which may reduce fairness. Increasing synchronization mechanisms to balance fairness and latency adds system complexity.

Simple synchronization reduces overhead but may cause starvation. Complex mechanisms improve fairness and predictability but increase processing cost. Designers must balance these trade-offs based on application requirements.

40. Explain how task synchronization is used in a real-time video streaming application.

In a real-time video streaming application, multiple tasks such as video capture, encoding, buffering, and transmission must be synchronized. Semaphores are used to signal frame availability, while mutexes protect shared buffers.

High-priority tasks handle real-time encoding and transmission to meet deadlines. Synchronization ensures smooth frame flow, prevents buffer overflow, and maintains video quality. Proper synchronization minimizes latency and ensures consistent real-time performance.

notes.arpanadhikari7.com.np

41. How does an automotive RTOS handle task synchronization for sensor data processing?

In automotive systems, an RTOS handles task synchronization carefully to ensure timely and reliable sensor data processing. Multiple sensors such as speed, temperature, pressure, radar, and camera sensors generate data simultaneously. The RTOS uses semaphores, mutexes, and event flags to synchronize tasks that read, process, and transmit sensor data.

Interrupts are generated when new sensor data arrives, and the Interrupt Service Routine (ISR) signals high-priority tasks using binary semaphores or event flags. Shared data buffers are protected using mutexes to prevent race conditions. Priority inheritance is used to avoid priority inversion when low-priority tasks access shared sensor resources. This synchronization ensures deterministic behavior, low latency, and safe data processing in automotive real-time environments.

Available on notes.arpanadhikari7.com.np

42. What is the difference between deadlock and starvation in an RTOS?

Basis	Deadlock	Starvation
Definition	Tasks wait indefinitely for resources held by each other	A task waits indefinitely due to low priority
Cause	Circular resource dependency	Continuous preemption by higher-priority tasks
System State	No progress possible	System progresses, but task never executes
Resource Availability	Resources are locked	Resources are available
Prevention	Resource ordering, avoidance protocols	Priority aging, fair scheduling
Severity	Critical system failure	Performance degradation

Deadlock completely halts progress, whereas starvation affects fairness and responsiveness.

43. In a robotic arm controlled by RTOS, how could deadlock arise during resource sharing (e.g., motors or sensors)?

Deadlock in a robotic arm system can arise when multiple tasks share motors, sensors, or communication buses. For example, one task may hold the motor control resource while waiting for sensor feedback, while another task holds the sensor interface and waits for motor access. This creates a circular wait condition.

If both tasks hold one resource and wait indefinitely for the other, deadlock occurs. This can freeze arm movement, stop sensor updates, and lead to unsafe behavior. Without proper deadlock prevention, the robotic system may fail to respond to control commands.

44. What precautions would you take to avoid deadlock when using shared hardware peripherals in embedded systems?

To avoid deadlock when using shared hardware peripherals, tasks should acquire resources in a fixed global order. Timeouts should be used when requesting peripherals so tasks do not wait indefinitely. Priority ceiling or priority inheritance protocols should be implemented to manage priority inversion.

Resources should be released as quickly as possible, and nested locking should be minimized. Where feasible, non-blocking communication and resource pooling should be used. These precautions ensure system reliability and uninterrupted operation.

45. In automotive systems, how might deadlocks arise during inter-task communication?

Deadlocks in automotive systems can arise during inter-task communication when tasks use blocking message queues or semaphores improperly. For example, one task may wait for a message while holding a mutex, while another task waits for that mutex before sending the message.

This circular dependency prevents both tasks from progressing. Such deadlocks can delay critical functions like braking, steering, or engine control. Proper communication design and avoidance of blocking calls inside critical sections are essential to prevent this issue.

Available on notes.arpanadhikari7.com.np

46. How does deadlock affect embedded systems with strict timing constraints, such as pacemakers or industrial controllers?

In systems with strict timing constraints, deadlock can have catastrophic consequences. Deadlocked tasks cannot execute, causing missed deadlines and system failure. In pacemakers, this could result in failure to deliver timely electrical pulses, risking patient safety.

In industrial controllers, deadlock can stop machinery, cause unsafe operating conditions, or lead to financial loss. Since real-time systems depend on predictable execution, deadlock directly violates real-time guarantees and must be strictly prevented.

47. Which deadlock prevention strategies (e.g., priority inheritance, timeout mechanisms) were applied, if any?

Deadlock prevention strategies commonly applied include priority inheritance, which prevents unbounded priority inversion but does not fully prevent deadlock. Timeout mechanisms are used to release resources if a task waits too long.

Priority ceiling protocols prevent circular wait by restricting resource access based on priority. Resource ordering ensures tasks acquire resources in a fixed sequence. In hard real-time systems, prevention is preferred over detection to maintain predictability.

48. What is critical section? What would happen if a lower-priority task preempted a critical higher-priority task?

A critical section is a part of code where shared resources are accessed and must not be interrupted to prevent data inconsistency. It is protected using mutexes, semaphores, or by disabling interrupts.

If a lower-priority task preempted a higher-priority task inside a critical section, it could cause data corruption, inconsistent system state, and missed deadlines. This violates real-time principles. RTOSs prevent this using priority inheritance or by disabling preemption within critical sections.

notes.arpanadhikari7.com.np

49. What strategies can be used to handle tasks that overrun their allocated execution time?

Tasks that overrun execution time can be handled using watchdog timers to detect and reset faulty tasks. Execution time monitoring helps identify overruns early. The scheduler may reduce the priority of overrunning tasks or terminate them.

Admission control prevents system overload by limiting task activation. In safety-critical systems, the RTOS may switch to a safe mode. These strategies ensure system stability and deadline compliance.

Available on notes.arpanadhikari7.com.np

50. How does RTOS ensure safe access to shared resources in a multithreaded environment?

An RTOS ensures safe access to shared resources using synchronization mechanisms such as mutexes, semaphores, event flags, and critical sections. Mutexes provide mutual exclusion and support priority inheritance to avoid priority inversion.

The RTOS scheduler enforces controlled access based on task priority and state. Atomic operations and proper interrupt handling further protect shared data. These mechanisms ensure data consistency, prevent race conditions, and maintain deterministic real-time behavior.

51. Describe the impact of task starvation and ways to prevent it in an RTOS.

Task starvation occurs when a task is unable to get CPU time for an extended period because higher-priority tasks continuously preempt it. In an RTOS, starvation mainly affects low-priority tasks and can lead to incomplete background operations, delayed system services, and degraded overall performance.

The impact of task starvation includes poor fairness, increased response time for non-critical tasks, and potential system instability. In long-running systems, starved tasks may never execute, leading to functional failures such as missed logging, communication breakdowns, or memory cleanup failures.

Ways to prevent task starvation include **priority aging**, where the priority of waiting tasks is gradually increased. **Time slicing** ensures that tasks of equal priority share CPU time fairly. Proper priority assignment and load balancing also reduce starvation. Some RTOSs use hybrid scheduling policies to maintain both real-time guarantees and fairness.

52. Compare fixed-priority preemptive scheduling with dynamic-priority scheduling in real-time systems.

Basis	Fixed-Priority Preemptive Scheduling	Dynamic-Priority Scheduling
Priority Assignment	Assigned at design time	Changes at runtime
Flexibility	Less flexible	Highly flexible
Predictability	Highly predictable	Less predictable
Overhead	Low	Higher
Deadline Handling	Suitable for static workloads	Adapts to varying deadlines
Examples	Rate Monotonic Scheduling (RMS)	Earliest Deadline First (EDF)
Usage	Hard real-time systems	Soft/firm real-time systems

notes.arpanadhikari7.com.np

Fixed-priority scheduling is simpler and predictable, while dynamic-priority scheduling provides better CPU utilization at the cost of higher complexity.

53. How are resources shared in Embedded Systems (ES)? Explain with an example. What are the trade-offs between using a mutex and a critical section for resource protection in RTOS?

Resources in embedded systems such as memory, I/O devices, sensors, and communication buses are shared using synchronization mechanisms like mutexes, semaphores, and critical sections. For example, if two tasks access a shared sensor buffer, a mutex is used to ensure only one task accesses it at a time.

Available on notes.arpanadhikari7.com.np

Trade-offs between mutex and critical section:

Aspect	Mutex	Critical Section
Scope	Task-level	Very short code blocks
Overhead	Higher	Very low
Priority Inheritance	Supported	Not supported
Blocking	Can block	No blocking
Usage	Resource sharing	Short atomic operations

Mutexes are safer for long resource usage, while critical sections are faster but must be kept very short.

54. Which one is better, multithreading or multitasking? What is multitasking? Explain the key characteristics of multitasking in ES?

Neither multithreading nor multitasking is universally better; suitability depends on application requirements. **Multitasking** refers to the ability of an operating system to execute multiple tasks concurrently by sharing CPU time.

In embedded systems, multitasking allows multiple functional tasks such as sensor reading, control processing, and communication to run concurrently.

Key characteristics of multitasking in ES include deterministic scheduling, priority-based execution, fast context switching, and efficient resource sharing. Multitasking improves responsiveness, modularity, and system scalability.

55. Differentiate between multiprocessing and multitasking. How does multiprocessing help in different areas?

Basis	Multitasking	Multiprocessing
CPU Count	Single CPU	Multiple CPUs
Execution	Time-shared	Parallel
Performance	Limited by CPU	Higher throughput
Cost	Lower	Higher
Complexity	Moderate	High

notes.arpanadhikari7.com.np

Multiprocessing helps in areas such as high-performance computing, industrial automation, data processing, and real-time analytics by enabling true parallel execution and higher reliability.

56. What is the difference between a binary semaphore and a mutex in resource sharing?

Aspect	Binary Semaphore	Mutex
Ownership	No ownership	Owned by task
Priority Inheritance	Not supported	Supported
Usage	Signaling	Resource protection
Risk	Priority inversion	Reduced inversion
Unlocking	Any task	Only owner task

Available on notes.arpanadhikari7.com.np

Mutexes are preferred for protecting shared resources, while binary semaphores are mainly used for synchronization.

57. How would you implement inter-task communication in an RTOS using message queues?

Inter-task communication using message queues involves creating a queue with a fixed size and message length. One task sends messages to the queue, while another receives them.

The RTOS manages queue access, ensuring thread safety and synchronization. Tasks can block while waiting for messages, reducing CPU usage. Message queues decouple sender and receiver tasks, improve modularity, and ensure safe data transfer between tasks.

58. How does priority inheritance solve priority inversion? Are there any drawbacks to this approach?

Priority inheritance temporarily raises the priority of a low-priority task holding a resource to match the highest-priority task waiting for it. This allows the low-priority task to complete quickly and release the resource.

Drawbacks include increased system complexity and temporary distortion of priority scheduling. It also does not prevent deadlock and adds runtime overhead.

59. How does an RTOS handle priority conflicts when multiple threads attempt to access a shared resource?

An RTOS handles priority conflicts using mutexes with priority inheritance or priority ceiling protocols. These mechanisms prevent priority inversion and ensure that high-priority tasks are not blocked indefinitely.

The scheduler enforces controlled access and may temporarily adjust priorities. Proper synchronization ensures predictable and safe resource sharing.

notes.arpanadhikari7.com.np

60. What are the risks of race conditions in multithreaded applications, and how does RTOS mitigate them?

Race conditions occur when multiple threads access shared data simultaneously without synchronization, leading to inconsistent or corrupted data.

RTOS mitigates race conditions using mutexes, semaphores, critical sections, and atomic operations. Proper scheduling and controlled access ensure data consistency and reliable execution.

Available on notes.arpanadhikari7.com.np

61. What are interrupts favored over polling? Distinguish between interrupt-driven and polling-based approaches.

Interrupts are favored over polling because they provide faster response and better CPU efficiency.

Aspect	Interrupt-Driven	Polling
CPU Usage	Efficient	Inefficient
Response Time	Fast	Slow
Power Consumption	Low	High
Complexity	Higher	Simpler
Usage	Real-time systems	Simple systems

62. How an interrupt is handled in an embedded system.

When an interrupt occurs, the CPU suspends the current task and saves its context. The processor jumps to the Interrupt Service Routine (ISR) using the interrupt vector table.

The ISR handles the event quickly and may signal a task. After completion, the CPU restores the previous context and resumes execution. RTOS ensures minimal latency and deterministic interrupt handling.

63. Explain the IPC mechanism in detail. Also mention the differences between their types.

Inter-Process Communication (IPC) mechanisms allow tasks to exchange data and synchronize execution. Common IPC mechanisms in RTOS include message queues, semaphores, mutexes, event flags, and shared memory.

IPC Type	Purpose	Data Transfer	Usage
Message Queue	Communication	Yes	Data passing
Semaphore	Synchronization	No	Signaling
Mutex	Mutual exclusion	No	Resource protection
Event Flags	Event signaling	Limited	Task coordination
Shared Memory	High-speed data	Yes	Large data exchange

notes.arpanadhikari7.com.np

IPC mechanisms ensure safe communication, synchronization, and predictable execution in real-time systems.

Available on notes.arpanadhikari7.com.np

Analytical Question 1: Autonomous Car Sensor System

Tasks:

- Obstacle detection every 10 ms
- Update navigation system every 50 ms
- Log sensor data every 200 ms

Questions and Answers:

a. Propose a suitable scheduling strategy (Rate Monotonic, Earliest Deadline First, etc.) and justify.

For this system, **Rate Monotonic Scheduling (RMS)** is suitable because it is a fixed-priority preemptive scheduling algorithm where tasks with shorter periods are assigned higher priority. Obstacle detection has the shortest period (10 ms), followed by navigation update (50 ms), and logging (200 ms). RMS ensures that the most time-critical tasks (like obstacle detection) are executed first, minimizing the risk of missing deadlines. Since all tasks have periodic execution with known deadlines, RMS provides deterministic and predictable behavior, which is essential in real-time automotive applications.

b. Explain how context switching occurs if a high-priority task preempts a running low-priority task.

When a low-priority task is running and a high-priority task becomes ready (e.g., obstacle detection), the RTOS scheduler triggers a **preemption**. The steps are:

1. The CPU saves the current context of the running low-priority task, including registers, program counter, and stack pointer.
2. The scheduler selects the high-priority task to run.
3. The high-priority task's context is loaded, and execution begins immediately.
4. After the high-priority task completes or blocks, the low-priority task's context is restored, allowing it to resume from where it left off.

This ensures high-priority tasks meet deadlines without losing the progress of lower-priority tasks.

notes.arpanadhikari7.com.np

c. If obstacle detection and navigation share sensor data, what synchronization mechanism would you use? Explain why.

A **mutex** would be the most appropriate synchronization mechanism. Both tasks need to safely access the shared sensor data without causing race conditions or data corruption. A mutex ensures **mutual exclusion**, meaning only one task can read/write the data at a time. Additionally, the mutex should support **priority inheritance** to prevent priority inversion, ensuring the high-priority obstacle detection task is not blocked indefinitely by the lower-priority navigation update task. This maintains data consistency and real-time responsiveness.

Available on notes.arpanadhikari7.com.np

Analytical Question 2: Factory Conveyor Robot

Tasks:

- Pick item every 20 ms
- Place item every 25 ms
- Update production statistics every 100 ms

Questions and Answers:

a. Identify possible scenarios where task synchronization is critical.

Task synchronization is critical when multiple tasks access **shared resources** such as conveyor belt sensors, robotic arm actuators, or production logs. Examples:

- Both “pick item” and “place item” tasks use the same robotic arm motors. If both try to move the arm simultaneously, mechanical conflict may occur.
- Updating production statistics while pick/place tasks are accessing shared counters or logs may lead to inconsistent values.

Synchronization ensures these shared resources are accessed safely without conflicts.

b. How can improper synchronization lead to data inconsistency?

Improper synchronization can result in **race conditions**. For instance:

- The pick task reads a counter while the place task is updating it. The counter may be incremented incorrectly.
- Two tasks update production statistics simultaneously, leading to lost or overwritten data.

Consequences include inaccurate production counts, faulty control signals, and potential system failures.

c. Explain the effect of frequent context switching on CPU utilization.

Frequent context switching occurs when high-priority tasks preempt lower-priority tasks repeatedly. Effects include:

- **Increased overhead**, as saving and restoring task contexts consumes CPU cycles.
- **Reduced CPU efficiency**, because more time is spent on switching rather than actual task execution.
- **Potential cache misses**, as CPU registers and memory states change frequently.

Although preemption ensures real-time deadlines are met, excessive context switching can degrade system throughput and may require optimization of task periods and priorities.

Available on notes.arpanadhikari7.com.np

Analytical Question 3: Multi-Robot Assembly Line

Tasks:

- Robot A: Weld parts (needs shared power tool)
- Robot B: Paint parts (needs the same power tool)
- Monitor and log process

Questions and Answers:

a. Describe a situation where deadlock can occur with the shared tool.

Deadlock can occur when both Robot A and Robot B are waiting indefinitely for the shared power tool while holding other resources. For example, Robot A acquires the power tool and starts welding, but before completing its task, it waits for confirmation or logging service that is currently being handled by the monitoring task. At the same time, Robot B may be holding a logging or status resource and waiting for the power tool to start painting. Since Robot A is waiting for a resource held by Robot B, and Robot B is waiting for the power tool held by Robot A, neither robot can proceed. This circular waiting condition results in a deadlock, causing the entire assembly line to halt. Deadlock leads to reduced productivity and can cause serious delays in real-time industrial systems.

b. Explain how priority inversion can happen if Robot A has a higher priority than Robot B.

Priority inversion occurs when a lower-priority task temporarily blocks a higher-priority task. In this case, Robot A (high priority) requires the shared power tool, but Robot B (lower priority) currently holds the tool. While Robot A waits, a medium-priority monitoring task may start executing repeatedly. Since Robot B has a lower priority, it does not get CPU time to release the power tool. As a result, Robot A remains blocked even though it has the highest priority. This situation inverts the intended priority order, where the high-priority task is indirectly delayed by lower-priority tasks, leading to missed deadlines in real-time operations.

c. Suggest RTOS mechanisms (mutex, semaphore, priority inheritance) to prevent deadlock or priority inversion.

notes.arpanadhikari7.com.np

To prevent deadlock and priority inversion, the RTOS should use a **mutex with priority inheritance**. A mutex ensures that only one robot can access the shared power tool at a time, preventing simultaneous access conflicts. Priority inheritance temporarily raises the priority of Robot B (lower priority) to match Robot A when Robot A is waiting for the tool. This allows Robot B to finish its task quickly and release the resource, after which Robot A resumes execution. Additionally, careful resource ordering and timeout mechanisms can be applied to reduce deadlock risk. These RTOS mechanisms ensure safe resource sharing, predictability, and real-time correctness.

Available on notes.arpanadhikari7.com.np

Analytical Question 4: Printing System in Office RTOS

Tasks:

- Print job processing (high priority)
- Document scanning (medium priority)
- Logging and status update (low priority)

Questions and Answers:

a. What happens if a low-priority logging task locks the printer resource?

If the low-priority logging task locks the printer resource, the high-priority print job processing task will be forced to wait until the logging task releases the resource. During this waiting period, medium-priority tasks such as document scanning may continue executing. As a result, the high-priority print task experiences unnecessary delays despite its importance. This situation can cause missed deadlines, slow printing response, and poor system performance, which is unacceptable in a real-time office printing system.

b. Explain how priority inheritance can resolve delays for high-priority tasks.

Priority inheritance resolves this problem by temporarily elevating the priority of the low-priority logging task when it holds a resource needed by a high-priority task. When the print job processing task requests the printer resource, the RTOS boosts the logging task's priority to that of the print task. This prevents medium-priority tasks from preempting it, allowing the logging task to complete quickly and release the printer resource. Once released, the logging task's priority returns to normal, and the high-priority print task resumes execution without excessive delay.

c. How does resource sharing affect overall system determinism?

Resource sharing introduces uncertainty in task execution timing, which directly affects system determinism. When multiple tasks compete for shared resources, blocking and waiting times can vary depending on execution order. Without proper synchronization mechanisms, this variability can lead to unpredictable response times and missed deadlines. However, using RTOS features such as mutexes, bounded blocking, and priority inheritance helps control these delays. Properly managed resource sharing ensures predictable behavior, maintaining determinism essential for real-time systems.

Available on notes.arpanadhikari7.com.np

Analytical Question 5: Drone Flight Controller

Tasks:

- Thread 1: Read gyroscope/accelerometer every 5 ms
- Thread 2: Update motor speeds immediately based on Thread 1 data
- Thread 3: Log flight data every 50 ms
- Thread 4: Send telemetry every 100 ms

Questions and Answers:

a. Explain the difference between multithreading and multitasking in this scenario.

In this drone flight controller, **multithreading** refers to multiple threads (Thread 1, Thread 2, Thread 3, Thread 4) executing within the same process and sharing the same memory space. These threads cooperate closely, especially where Thread 2 immediately uses sensor data produced by Thread 1. Context switching between threads is faster because memory is shared.

Multitasking, on the other hand, refers to the RTOS scheduling multiple independent tasks or processes. If each function (sensor reading, motor control, logging, telemetry) were implemented as separate RTOS tasks with isolated resources, it would be multitasking. Multitasking involves higher overhead due to separate task stacks and context switching.

In this scenario, multithreading is preferred because the tasks are tightly coupled, time-critical, and require fast data sharing.

b. How can multithreading improve CPU utilization and responsiveness?

Multithreading improves CPU utilization by allowing the processor to switch to another thread when one thread is waiting for I/O or a timer event. For example, while Thread 3 is logging data or Thread 4 is sending telemetry, the CPU can execute Thread 1 or Thread 2 instead of remaining idle.

Responsiveness is improved because high-priority threads such as Thread 2 (motor speed update) can preempt lower-priority threads immediately. This ensures rapid reaction to sensor data changes, which is critical for flight stability. Efficient context switching and shared memory further reduce delays, making the drone control system more responsive and real-time compliant.

c. What issues could arise if Thread 3 blocks Thread 2? How can RTOS handle it?

If Thread 3 (logging task) blocks Thread 2 (motor control), the drone may fail to update motor speeds in time. This can cause unstable flight, loss of control, or even a crash. Such blocking may occur due to improper synchronization, shared resource locking, or priority inversion.

The RTOS can handle this using **priority-based scheduling and priority inheritance**. Thread 2 should be assigned the highest priority. If Thread 3 holds a shared resource needed by Thread 2, priority inheritance temporarily boosts Thread 3's priority so it can release the resource quickly. Additionally, using mutexes instead of semaphores and minimizing critical sections ensures that real-time motor control is never delayed by non-critical tasks.

Available on notes.arpanadhikari7.com.np

Analytical Question 6: Smart Home Controller

A smart home embedded system handles:

- Temperature sensing
- Light control
- Wi-Fi communication
- Voice command processing

Sometimes voice commands are delayed.

Scenario Questions and Answers:

a. Which multitasking issue causes command latency?

Command latency is mainly caused by **improper task prioritization and task starvation**. If background tasks such as temperature sensing or Wi-Fi communication consume excessive CPU time, the voice command processing task may not get scheduled immediately. Long blocking calls, poor synchronization, or excessive context switching can also delay the execution of voice commands, leading to noticeable response lag for the user.

b. How can priority-based scheduling solve this?

Priority-based scheduling assigns higher priority to time-critical tasks such as voice command processing. When a voice command is detected, the RTOS immediately preempts lower-priority tasks like sensing or background communication and executes the command task. This ensures fast response and improved user experience. Proper priority assignment also prevents command starvation and ensures deterministic behavior in the system.

c. Should communication tasks run at a higher priority than sensing tasks? Explain.

Yes, communication tasks such as Wi-Fi handling should generally run at a higher priority than periodic sensing tasks. Communication tasks often involve external interactions and time-sensitive data transfer, such as sending voice commands to a cloud service. Delaying communication can cause command timeouts or poor system responsiveness. In contrast, sensing tasks usually tolerate small delays without affecting system functionality. Therefore, assigning higher priority to communication tasks ensures reliable and responsive system operation.

Available on notes.arpanadhikari7.com.np

7. Smart Traffic Light System
A traffic light controller performs:

- Signal timing control
- Pedestrian button handling
- Traffic density analysis

Scenario Questions and Answers:

a. Which parts should use multitasking and which should use multithreading?

In a smart traffic light system, **multitasking** should be used for **signal timing control, pedestrian button handling, and traffic density analysis** because these functions are logically independent and can be implemented as separate RTOS tasks. Each task has its own timing requirements and priority levels, making task-based multitasking suitable.

Multithreading can be used *within* the traffic density analysis task, where multiple threads may process camera input, sensor fusion, or data filtering in parallel. These operations share the same data and memory space and benefit from fast inter-thread communication.

Thus, multitasking ensures system-level separation and real-time control, while multithreading improves efficiency within complex computation tasks.

b. Why is multitasking preferred for signal control?

Multitasking is preferred for signal control because traffic signal timing is a **hard real-time task** that must meet strict deadlines. Each signal phase (red, yellow, green) must be executed deterministically without interference from other system activities.

By assigning signal control as a high-priority RTOS task, multitasking ensures that it can preempt less critical tasks such as traffic density analysis or logging. This guarantees predictable timing behavior, fault isolation, and system reliability. Multitasking also simplifies scheduling analysis and makes it easier to verify real-time constraints.

notes.arpanadhikari7.com.np

c. What are the risks of excessive threads in embedded systems?

Excessive threads in embedded systems increase **memory usage**, as each thread requires its own stack. This can quickly exhaust limited RAM. Frequent context switching between many threads increases CPU overhead, reducing real-time responsiveness.

Additionally, excessive threads complicate synchronization, increasing the risk of race conditions, deadlocks, and priority inversion. Debugging also becomes more difficult. In real-time embedded systems, uncontrolled thread creation can compromise predictability and timing guarantees, making excessive threading risky.

Available on notes.arpanadhikari7.com.np

8. Wearable Health Monitoring Device
The device monitors:

- Heart rate
- Body temperature
- Bluetooth transmission

The system has limited RAM and power.

Scenario Questions and Answers:

a. Should the system use multitasking or multithreading?

The wearable health monitoring device should primarily use **multitasking** rather than multithreading. Each function—heart rate monitoring, temperature sensing, and Bluetooth transmission—can be implemented as a separate RTOS task with clearly defined priorities and execution periods.

Multithreading is less suitable because shared memory increases complexity and synchronization overhead. In a resource-constrained wearable device, simpler task-based multitasking ensures better control over memory usage, timing behavior, and power consumption.

b. How do resource constraints affect design choice?

Limited RAM restricts the number of tasks and stack sizes that can be allocated. Limited CPU capability requires minimizing context switches and avoiding unnecessary computations. Power constraints demand that the CPU remain in low-power sleep states as much as possible.

These constraints push the design toward fewer tasks, minimal synchronization mechanisms, static memory allocation, and efficient scheduling. Designers must carefully balance responsiveness and resource usage, often preferring simpler multitasking models over complex multithreaded designs.

c. What scheduling policy is suitable for battery efficiency?

notes.arpanadhikari7.com.np

A **fixed-priority preemptive scheduling policy with tickless idle support** is most suitable for battery efficiency. Periodic sensing tasks (heart rate and temperature) can run at lower priorities with well-defined intervals, while Bluetooth transmission runs only when required.

Tickless scheduling allows the CPU to sleep longer between task executions, significantly reducing power consumption. Assigning priorities based on urgency ensures that the system remains responsive while maximizing battery life, which is critical for wearable devices.

Available on notes.arpanadhikari7.com.np